



PDF

helpmeet implement this project ill give you the readme from the GitHub they've given me and I need to implement this tell me step by step

Number Guessing Repair Lab

This project is an interactive number deduction game using Pygame. It introduces students to string-to-integer parsing safety, boundary and input sanitization, dynamic range feedback, and text-input UI widgets within an object-oriented codebase.

What's Provided

A working Number Guessing game with:

- A secret number generated randomly between 1 and 100 on game initialization and resets
- A custom `TextBox` input component handling digit typing, backspace, and active selection states
- Interactive guess submission via the `Return / Enter` key or clicking the `SUBMIT` button
- Dynamic hint feedback (`TOO LOW!`, `TOO HIGH!`, or `CORRECT!`) along with an attempt counter
- Win state handling and restart capability via the `R` key

It has one deliberate bug and three optional features left as tasks to implement. You are expected to analyze, interact with an AI assistant, and complete/fix the game to make it fully functional and more interesting.

Use an LLM (e.g. ChatGPT or Claude) as your debugging and pair-programming partner for this lab.

Getting Started

Setup

1. Make sure you have Python 3.10+ installed.
2. Install dependencies:

```
pip install pygame
```

3. Run the game:

```
python main.py
```

Controls: Type digits into the text box and press `Return` (or click `SUBMIT`) to guess. Press `R` to start a new game after winning.

Tasks to Complete

Each task must be completed using an iterative process involving LLM suggestions and your critical code review.

Task 1: Fix the empty input crash bug

Submitting a guess without typing any numbers causes the application to crash immediately with a value parsing error. Ensure that empty submissions are safely handled with a warning prompt without terminating the program or consuming an attempt.

Task 2: Implement dynamic search range display

The game currently only provides one-off high or low indicators without keeping track of the narrowing search window. Track the valid minimum and maximum boundaries established by previous guesses and display the narrowed range on screen to guide the player's next move

Task 3: Implement Recent Guess History Tracker

Players have no visual record of previous numbers they have already tested. Add a history log panel displaying the last several guesses alongside color-coded directional indicators showing whether each was too high or too low.

Task 4: Implement maximum Attempts Constraint & Failure State

Players currently enjoy unlimited guesses, removing stakes from deduction.

Introduce a strict attempt limit that triggers a Game Over screen revealing the hidden number if the player runs out of tries.

Expected Behavior

- Typing numbers into the text box and pressing Return or clicking SUBMIT registers a guess.
- Submitting an empty input box displays a warning message without crashing the game.
- The game accurately indicates whether the guess is too high or too low, incrementing the attempt count each time.
- Guessing the exact number displays the victory message and unlocks the R key restart option.

Folder Structure

```
number_guess/  
├── game/  
│   ├── game_engine.py  
│   └── text_box.py  
├── main.py  
└── README.md
```

Submission Checklist

Submission is only the following three things:

- [] A 10-second video of gameplay before your changes, showing the bug/broken behavior
- [] A 10-second video of gameplay after your changes, showing the bug fixed and the new features working
- [] The Chat/LLM used page link, with the complete chat history

Troubleshooting a game engine without access to its source files.

Troubleshooting a game engine without access to its source files.

Since your lab is graded on the chat history too, the cleanest approach is to do the coding in this conversation and export it at the end. Here's the full plan.

Phase 1: Setup (about 10 min)

1. **Fork the repo** on GitHub into your personal account. Don't raise a PR to SETAPESU26.

2. **Clone your fork:**

```
git clone https://github.com/<your-username>/<repo-name>.git
```

```
cd <repo-name>
```

3. **Install and run:**

```
pip install pygame
```

```
python main.py
```

4. **Record the "before" video (10 sec).** Type a guess or two, then press Enter with an empty box so the crash is on camera. On Windows, Win+Alt+R starts Game Bar recording. On a Mac, use Cmd+Shift+5. Save it as before.mp4.

Phase 2: Fix and add features (about 35 min)

Do one task at a time, test it, then commit it before starting the next:

Task	What changes	Commit message
1. Empty input crash	Check for empty text before <code>int()</code> , show a warning, don't count the attempt	Fix empty input crash
2. Search range display	Track <code>low</code> <code>high</code> bounds (start 1–100), narrow them after each guess, draw "Range: X – Y"	Add dynamic search range display
3. Guess history	Keep a list of <code>(guess, result)</code> , draw the last ~5 in a side panel, red for too high and blue for too low	Add recent guess history panel
4. Attempt limit	Set <code>MAX_ATTEMPTS</code> (e.g. 7), show a Game Over screen with the secret number, allow R to restart	Add max attempts and game over state

After each task, run:

```
git add .
```

```
git commit -m "<message from table>"
```

Phase 3: Submit (about 10 min)

1. **Record the "after" video (10 sec).** Show an empty submit with the warning, a few guesses so the range and history update, and ideally a Game Over screen.
2. **Create the Lab-4 folder** in your Lab repo with the videos, the updated code, and this chat exported as a PDF.
3. **Push:**

```
git push origin main
```

What I need from you now

The fixes have to match how your code is actually written, so paste the full contents of these three files:

- `game/game_engine.py`
- `game/text_box.py`
- `main.py`

Once I see them, I'll give you the exact Task 1 fix first. It's almost certainly an `int(self.text_box.text)` call with no check for empty input. Then we'll go through Tasks 2 to 4, one prompt each.

tell me from the set up I have the GitHub link opened on my safari where ive logged in. do the entire project for me

Rebuilding a complete project from scratch without access to the repository.

Rebuilding a complete project from scratch without access to the repository.

I can write all the code, but I can't see your GitHub or open Safari, so I need the starter code from you. First, do the setup on your Mac. It takes about 5 minutes.

Step 1: Fork the repo (in Safari)

1. On the repo page, click **Fork** (top right).
2. Keep the default name and click **Create fork**.
3. You're now on *your copy* (the URL will show your username). Click the green **Code** button and copy the HTTPS link.

Step 2: Clone and run (in Terminal)

Open Terminal (Cmd+Space, type "Terminal") and run these one at a time, pasting your link where shown:

```
cd ~/Desktop
```

```
git clone PASTE_YOUR_LINK_HERE
```

```
cd PASTE_THE_FOLDER_NAME_HERE
```

```
pip3 install pygame
```

```
python3 main.py
```

If pip3 gives an "externally managed environment" error, use `pip3 install pygame --break-system-packages` instead.

Step 3: Record the "before" video

1. With the game open, press **Cmd+Shift+5**, choose **Record Selected Portion**, drag over the game window, and click **Record**.
2. Type a number and press Enter a couple of times, then press Enter with the box **empty** so the crash shows.
3. Stop recording from the menu bar. The video saves to your Desktop. Rename it `before.mov`.

Step 4: Send me the code

Paste the contents of these three files here (open each in TextEdit or VS Code, select all, copy):

- `game/game_engine.py`
- `game/text_box.py`
- `main.py`

Or paste the link to your fork and I'll try to read it directly.

Once I have the code, I'll send back the finished versions one task at a time. After each one, you paste it in, test it, and run the commit command I give you, so you end up with the 4 separate commits the lab requires. At the end, I'll walk you through the "after" video, the push, and exporting this chat as a PDF.

Checking the correct repository for the upload.

Checking the correct repository for the upload.

Not yet. Two things are off:

1. **This is the wrong repo.** The top says `60_number_guess`, which is your fork, and that's already done. The Lab-4 folder goes in your separate **Lab repo**.
2. **No files have been added.** The box still says "Drag files here."

Click **Cancel**. Nothing has been changed.

Then:

1. Open your **Lab repo**. You have a tab open called `undynelearns/S...`, which might be it. Click that tab and check whether it's your lab submissions repo. If you're not sure, send a screenshot.
2. On the Lab repo's main page, click **Add file**, then **Upload files**.
3. Open Finder, go to the **Desktop**, and drag the **Lab-4** folder itself into the "Drag files here" box.
4. Wait until you see all 6 files listed below the box, each showing a path starting with `Lab-4/`.
5. Leave "**Commit directly to the main branch**" selected, which is already the default. Don't pick the pull request option.
6. Type `Add Lab-4 submission` and click **Commit changes**.

Send a screenshot after step 4 if you'd like me to check before you commit.



Screenshot 2026-10-07 at 12.44.26 PM.png



Screenshot 2026-10-07 at 12.44.39 PM.png



Screenshot 2026-10-07 at 12.44.46 PM.png

check if everything's done?

Yes, the upload is correct. Your Lab repo SELABS_PES1UG24CS927_5B now has a **Lab-4** folder containing:

- before.mov and after.mov
- the updated code: main.py, README.md, and game/ with both game_engine.py and text_box.py
- chat_history.pdf

Your fork is also pushed, with 4 separate Task commits and no pull request to SETAPESU26.

The one thing left is the chat PDF. It was 24 KB and saved before these last few messages, so it may be incomplete. Click chat_history.pdf on GitHub to check it. If it's blank or cut off, fix it like this:

1. In the Claude tab, scroll from the very top to the bottom so everything loads.
2. In Safari, choose **File**, then **Export as PDF**, and save it to your Desktop as chat_history.pdf.
3. In your Lab repo, open the **Lab-4** folder, click **Add file**, then **Upload files**, and drag in the new PDF. A file with the same name replaces the old one.
4. Commit with the message Update chat history.

Also click **Share** at the top of this chat and keep the link handy, since the README checklist asks for the chat link.

Once the PDF checks out, you're finished.

[Claude is AI and can make mistakes. Please double-check responses.](#)

